SecureDT - Google Drive (Cloud) Setup Guide

Step-by-step procedure for your machine | Generated: 2026-09-13 17:54:54
SecureDT | Intelligent and Secure Task Scheduling for Digital Twin-Enabled Edge-Cloud Networks

0. Where the Google Drive config lives

There are exactly **4 places** involved in adding Google Drive cloud storage. Three are on YOUR PC, one is on Google's side:

[1] Google Cloud Console (Google website) - you create the project, enable the Drive API, and make a Service Account. You download a JSON key here.

[2] Project credentials folder - `SecureDT\credentials\service-account.json` (folder must be created - it does not exist yet).

[3] .env file - `SecureDT\.env`. Two new lines tell Django where the JSON key is and which Drive folder to use. **This is the file you already updated for MySQL (SecureDT/SecureDT).**

[4] Google Drive website - you create the storage folder and share it with the service account email.

At the end, Django reads `settings.GOOGLE_DRIVE_FOLDER_ID` and `settings.GOOGLE_SERVICE_ACCOUNT_FILE` from your .env (see `securedt\settings.py` lines 178-179). The cloud app (cloud\google_drive_service.py) loads the JSON automatically. Until then the Cloud page runs in safe SIMULATION mode.

Step 1 - Create a Google Cloud project

1. Open https://console.cloud.google.com/ in your browser (sign in with the Google account that owns your Drive).
2. Click the project dropdown at the top-left and choose **New Project**.
3. Name it **SecureDT-Project** (name is optional).
4. Click **Create**. Select it from the dropdown when done.

Step 2 - Enable the Google Drive API

1. In the left menu go to **APIs & Services** > **Library**.
2. Search for **Google Drive API**.
3. Open it and click **Enable**.

Step 3 - Create the Service Account

1. Left menu: **IAM & Admin** > **Service Accounts** > **Create Service Account**.
2. Name: **securetd-drive-service** | Service account ID: **securetd-drive-service**.
3. Click **Create and Continue**.
4. Role: choose **Editor** (simplest, recommended here). Then **Done**. Note the service account **email** - it looks like `securetd-drive-service@SECUREDT-PROJECT.iam.gserviceaccount.com`. You need it in Step 6.

Step 4 - Download the JSON key

1. In the Service Accounts list, click the email you just created.
2. Go to the **Keys** tab > **Add Key** > **Create New Key**.
3. Choose **JSON** and click **Create**. A `xxx-key.json` file downloads.
4. **IMPORTANT:** this file is the only credential Django will ever have. Keep it private, never share it, and never commit it to git (it is already in .gitignore area).

Step 5 - Save the key in the project

1. In Windows Explorer open the project folder:

```
C:\Users\mkail\capstone project\project vs code\SecureDT
```

2. Create a new folder named **credentials** inside it.
3. Rename the downloaded JSON to **service-account.json** and move it into the credentials folder.
4. Result should be exactly:

```
SecureDT\credentials\service-account.json
```

Step 6 - Create the Drive folder and share it

1. Open https://drive.google.com/ .
2. Click **New** > **New folder** and name it **Intelligent_Secure_Scheduling**.
3. Right-click the folder > **Share** > add the service account email from Step 3.
4. Set permission to **Editor** and click **Send**.
5. Copy the **Folder ID**: open the folder in the browser - the URL looks like:

```
https://drive.google.com/drive/folders/1AbCdeFgHiJkLmNoPqRsTuVwXyz
```

The part after `/folders/` is the Folder ID (e.g. `1AbCdeFgHiJkLmNoPqRsTuVwXyz`).

Step 7 - Add the two lines to .env (where the config goes)

Open `SecureDT\.env` in VS Code and add these lines (or paste under the existing MySQL section - they do not exist in this file yet):

```
GOOGLE_SERVICE_ACCOUNT_FILE=./credentials/service-account.json
```

```
GOOGLE_DRIVE_FOLDER_ID=1AbCdeFgHiJkLmNoPqRsTuVwXyz
```

Replace the example Folder ID with the one from Step 6. Your .env should now look like this (MySQL section unchanged):

```
AI_PROVIDER=localfeatures
HOST=127.0.0.1
PORT=8000
DEBUG=True
MYSQL_DATABASE=securetdb
MYSQL_USER=SecureDT
MYSQL_PASSWORD=SecureDT
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3307
GOOGLE_SERVICE_ACCOUNT_FILE=./credentials/service-account.json
GOOGLE_DRIVE_FOLDER_ID=1AbCdeFgHiJkLmNoPqRsTuVwXyz
```

Step 8 - Restart and verify

1. Restart the Django server (Ctrl+C then `python manage.py runserver` in the SecureDT folder) so it picks up the new .env values.

2. Check the Cloud / Storage page in the UI, or call the status endpoint:

```
GET http://127.0.0.1:8000/api/cloud/status/
```

3. It should show **GOOGLE_DRIVE** mode (not SIMULATION). The Django app auto-creates subfolders (datasets, results, models, etc.) in your Drive folder on first use.

4. Upload any file via the UI - it should appear inside **Intelligent_Secure_Scheduling** on Google Drive.

Security reminders

- MEC agents never see these Google credentials; all Drive traffic flows through the Django API proxy.

- Keep service-account.json out of git and backups you do not control.

- If something fails, the Cloud module falls back to SIMULATION mode gracefully - the rest of the app keeps working.